Tutorial II.I - Variables and Types

Applied Optimization with Julia



Introduction

Welcome to this interactive Julia tutorial which introduces the basics of variables and types. Understanding variables and their types is crucial as they are the building blocks of any program. They determine how data is stored, manipulated, and how efficiently your code runs.

This script is designed to be interactive. Follow the instructions, write your code in the designated code blocks, and then execute the corresponding code. Each exercise is followed by an `@assert` statement that checks your solution.

Julia in Zed

1. First, install the **Julia** extension for syntax support: open the command palette (`Ctrl+Shift+P`, macOS `Cmd+Shift+P`), choose **zed: extensions**, and search for "Julia".
2. Create (or pick) the folder where you want to save all the files for this course.
3. Open a terminal in that folder and run:

   ```
   julia --project=applied-optimization
   ```

   This starts Julia with the course project environment.
4. In the Julia REPL that opens, run:

   ```
   using Pkg
   Pkg.add("IJulia")  # Add IJulia to the environment
   ```

5. Still in Julia, run:

   ```
   using IJulia
   zed_plots = raw"""
   push!(Base.package_callbacks, pkg -> pkg.name == "Plots" &&
       @eval Base.showable(::MIME"text/html", ::
   $(Base.loaded_modules[pkg]).Plot) = false)
   """
   IJulia.installkernel("Applied Optimization", "--project=" *
   abspath("applied-optimization"), "-e", zed_plots)
   write(".JETLSConfig.toml", """
   [[diagnostic.patterns]]
   pattern = "(lowering|inference)/undef-global-var"
   match_by = "code"
   match_type = "regex"
   severity = "off"
   ```

```
path = "**/tutorial-*.jl"
""")
```

This creates a Jupyter kernel that uses the course environment: it is how Zed runs Julia code. The `zed_plots` part makes your plots visible in Zed: Zed cannot show the web version of a plot yet, so the kernel sends plots as images instead. The `write` line saves a settings file for Zed's Julia extension in your course folder. Without it, the extension underlines the test cells of every tutorial with "… is not defined", because they check variables that you only create when you solve the exercise.

6. Type `exit()` or press Ctrl+D to exit the Julia REPL. If Zed was open during these steps, quit it completely (macOS: `Cmd+Q`) and start it again: Zed only reads its kernels when it starts.
7. Download this tutorial as a `.jl` file from the course website and save it in the `applied-optimization` folder that the steps above created inside your course folder. You can do so by clicking on the "Julia" link in the sidebar (under "Code Links") on the tutorial page. Keep all tutorial files in that folder, together with the `data` folders some of them need later: Zed's Julia extension then knows the course packages.
8. In Zed, open your course folder (the one that contains `applied-optimization`, not `applied-optimization` itself) and open the `.jl` file from there: the settings file from step 5 only works while the course folder is open. The file is split into cells by `# %%` markers.
9. Put the cursor in a cell and press `Ctrl+Shift+Enter` (also on macOS), or run **repl: run** from the command palette. This runs the cell and jumps to the next one. Output appears right below the code. Zed picks the kernel by itself: the kernel selector at the top right of the editor should show the kernel that starts with **applied-optimization** — that is the kernel you just installed. If it shows another one (such as `julia-1.13`), click it and choose yours.

> ⚠ **Warning**
> Sorry that the start is rather complicated. But by following these steps, you get a clean environment to work in and you basically cannot break anything by installing packages.

> ℹ **Note**
> These steps ensure you're working in a clean Julia environment for the course. The `--project=applied-optimization` flag tells Julia to create (or later reuse) an environment in a subfolder called `applied-optimization`, keeping all course packages organized in one place and avoiding conflicts with other projects. The kernel remembers the exact Julia version it was installed with: close Zed before you update Julia (for example with `juliaup update`), then run steps 3 and 5 again.

> **Fallback: VS Code**
> If Zed does not run on your machine (for example, an older Windows GPU without DirectX 11), install VS Code with the Julia extension instead. Open the same `.jl` file there: the extension recognizes the `# %%` cell markers, and `Alt+Enter` (or `Shift+Enter`) runs the current cell in the Julia REPL using the course environment.

Section 1 - Variables

Think of variables as labeled containers. Just like you might label a box “Books” to store books, in programming we label our data with variable names. For example:

```
age = 30        # A box labeled "age" containing the number 30
```

```
30
```

```
name = "Tobias"  # A box labeled "name" containing the text "Tobias"
```

```
"Tobias"
```

Exercise 1.1 - Declare a Variable

Declare a variable named `x` and assign it the value `1`.

```
# YOUR CODE BELOW
```

```
# Test your answer
@assert x == 1 "Check again, the value of x should be 1. Remember to assign the value directly to x."
println("Great, you have correctly assigned the value $x to the variable 'x'.")
```

> **Note**
> Always replace ‘YOUR CODE BELOW’ with your actual code.

Exercise 1.2 - Declare a String Variable

Declare a variable named `hi` and assign it the string `"Hello, Optimization!"`.

```
# YOUR CODE BELOW
```

```
# Test your answer
@assert hi == "Hello, Optimization!" "Make sure the variable 'hi' contains the
```

```
exact string \"Hello, Optimization!\" - including the exclamation mark."
println("Good, the variable 'hi' now states \"$hi\".")
```

Section 2 - Basic Types

Just like real containers come in different types (like boxes for books, refrigerators for food, etc.), variables in Julia have different types depending on what they store:

- Integers (Int): Whole numbers like `1`, `42`, `-10`
- Floats (Float64): Numbers with decimal points like `3.14`, `-0.5`
- Booleans (Bool): True/false values like `true`, `false`
- Strings (String): Text in quotes like `"Hello"`

You can check what type of "container" a variable is using `typeof()`. Try this:

```
age = 25
typeof(age)     # Will show Int64 (integer type)
```

```
Int64
```

```
price = 19.99
typeof(price)   # Will show Float64 (decimal number type)
```

```
Float64
```

Exercise 2.1 - Create an Integer Variable

Create an Integer variable `answerUniverse` and set it to `42`.

```
# YOUR CODE BELOW
```

```
# Test your answer
@assert answerUniverse == 42 && answerUniverse isa Int "The variable
'answerUniverse' should hold the Integer 42. Note that 42.0 would be a Float,
not an Integer - you can check with typeof(answerUniverse)."
println("Great, the answer to all questions on the universe is $answerUniverse
now.")
```

Exercise 2.2 - Create a Float Variable

Create a Float variable `money` and set it to `1.35`.

```
# YOUR CODE BELOW
```

```
# Test your answer
@assert money == 1.35 && money isa Float64 "The variable 'money' should hold
the Float64 1.35. Remember the decimal point - you can check the type with
typeof(money)."
println("Perfect, you have stored $money in the variable 'money'.")
```

Exercise 2.3 - Create a Boolean Variable

Create a Boolean variable `isStudent` and set it to `true`.

```
# YOUR CODE BELOW
```

```
# Test your answer
@assert isStudent isa Bool && isStudent == true "The variable 'isStudent'
should be the Boolean value true - not the number 1."
println("Correct, you are a student now.")
```

Section 3 - Type Annotations and Inference

Sometimes we want to specify exactly what kind of "container" we want to use. In Julia, we can do this using type annotations. Why should you care? Later in the course, JuMP will insist on the right kinds of numbers for your optimization models, and telling Julia the exact type also helps it run your code fast.

```
temperature::Float64 = 37.0    # Specifically saying we want a decimal number
```

```
37.0
```

```
n_items::Int64 = 100           # Specifically saying we want a whole number
```

```
100
```

Exercise 3.1 - Type Annotation

Declare a variable `y` with an explicit type annotation of `Int64` and assign it the value `5`.

```
# YOUR CODE BELOW
```

```
# Test your answer
@assert y == 5 && typeof(y) == Int64 "Make sure 'y' is of type Int64 and has
the value 5."
println("Great! You've created an Int64 variable 'y' with the value $y.")
```

Section 4 - String Interpolation

String interpolation is like filling in blanks in a sentence. Instead of writing:

```
name = "Tobias"
age = 30
# The hard way:
message = "My name is " * name * " and I am " * string(age) * " years old"
```

```
"My name is Tobias and I am 30 years old"
```

We can use the `$` symbol to insert variables directly into our text:

```
message = "My name is $name and I am $age years old"
```

```
"My name is Tobias and I am 30 years old"
```

It's like having a template where Julia automatically fills in the values for you! If you have paid attention to the previous exercise, you have already seen this in action. The following example illustrates this again:

```
language = "Julia"
println("I'm learning $language")
```

```
I'm learning Julia
```

Exercise 4.1 - String Interpolation

Create a string `message` that says `"y is [value of y]"` using string interpolation. This uses the variable `y` you created in Exercise 3.1 - if you get an `UndefVarError`, run that exercise again first.

```
# YOUR CODE BELOW
```

```
# Test your answer
@assert message == "y is 5" "Make sure your string includes the correct value
of y."
println("Excellent! Your interpolated string is: $message")
```

Conclusion

Congratulations! You have completed the first tutorial on Variables and Types. You've learned about the basics of variables, integers, floats, booleans, and strings. Continue to the next file to learn more.

Solutions

You will likely find solutions to most exercises online. However, I strongly encourage you to work on these exercises independently without searching explicitly for the exact answers to the exercises. Understanding someone else's solution is very different from developing your own. Use the lecture notes and try to solve the exercises on your own. This approach will significantly enhance your learning and problem-solving skills.

Remember, the goal is not just to complete the exercises, but to understand the concepts and improve your programming abilities. If you encounter difficulties, review the lecture materials, experiment with different approaches, and don't hesitate to ask for clarification during class discussions.

Bibliography